

Milestone 2 Completion Report

Project Title: BankBot - An AI Chatbot for Banking FAQs

Milestone Title: Milestone 2: Dialogue Logic & State Management

Intern Name: [Author]

Date : November 15, 2025

1. Executive Summary

Following the successful development of the Natural Language Understanding (NLU) engine in Milestone 1, the BankBot project has advanced to the **Dialogue Management** phase. While Milestone 1 established the system's ability to classify intents (e.g., identifying a user wants to "transfer money"), it lacked the executive function to manage multi-turn conversations or retain context.

In Milestone 2, we have successfully implemented a **Hybrid Dialogue Manager**. This system integrates the probabilistic predictions of the Machine Learning model with a deterministic State Machine. This architecture enables the bot to handle complex, non-linear workflows—such as nested menu navigation for loan inquiries and context-aware slot filling for fund transfers—ensuring a robust and user-friendly banking experience.

2. Technical Architecture

The core of Milestone 2 is the transition from a stateless "Question-Answer" model to a stateful "Conversational Agent." The architecture is defined by three distinct layers of logic.

2.1 The Hybrid Logic Core

To maximize reliability, the `dialogue_manager.py` utilizes a tiered logic approach:

- High-Priority Context Layer (The "Ears"):** Before processing new text, the system checks the user's current **State**. If the bot is waiting for specific data (e.g., an account number), it bypasses the NLU to prevent misclassification and uses Regex to capture raw input.
- Keyword Override Layer:** To address NLU weaknesses with short-text inputs (e.g., "Hi", "Loan"), a deterministic `KEYWORD_MAP` forces specific intents, ensuring 100% accuracy for common commands.

3. **NLU Inference Layer:** If no state or keyword is active, the system calls the Milestone 1 Spacy model (`models/model-best`) to predict the intent and extract entities.
4. **Fallback Layer:** If the NLU confidence is low, the system falls back to a static FAQ lookup (CSV) or a graceful error message.

2.2 Finite State Machine (FSM)

We implemented a custom class, `BankBotLogic`, which maintains a session dictionary for the user. This creates "memory" across conversation turns.

- **IDLE:** The default state where the bot listens for new commands.
- **WAITING_FOR_MENU_SELECTION:** The bot restricts valid inputs to the current menu options (e.g., Debit vs. Credit).
- **WAITING_FOR_TRANSFER_PERSON / AMOUNT:** The bot "locks" onto a specific slot-filling task until the required information is provided.

2.3 Nested Menu System

To handle complex queries like "Card Services" without overwhelming the user, we implemented a dictionary-based decision tree (`MENU_FLOWS`). This allows for deep navigation:

- *Level 1:* Card Services
 - *Level 2:* Debit Card vs. Credit Card
 - *Level 3:* Block vs. Limit vs. New
-

3. Key Features Implemented

3.1 Context Retention & Slot Filling

The system can now handle fragmented information. In a "Transfer Money" scenario, the bot identifies missing variables and prompts the user specifically for them.

Logic Implemented:

Python

if not person:

```
session["state"] = "WAITING_FOR_TRANSFER_PERSON"
```

```
return "💎 Who is the recipient?"
```

if not amount:

```
session["state"] = "WAITING_FOR_TRANSFER_AMOUNT"
```

```
return "💎 How much do you want to send?"
```

-

3.2 Smart Fallbacks & Input Validation

A major improvement over standard NLU implementations is the handling of raw data.

- **Regex Integration:** If the bot asks for an account number, and the user types "12345", the NLU might misclassify this as a date or a quantity. Our logic intercepts this input using `re.search(r'\d+', text)` to ensure the data is captured correctly regardless of NLU prediction.

3.3 Dynamic Keyword Mapping

To resolve the "Single Word Failure" issue common in ML models (where a single word like "Card" lacks enough context for high confidence), we implemented a pre-processing map:

- "loan" → Forces `loan_inquiry`
- "balance" → Forces `check_balance`
- "hi" → Forces `greet`

4. Testing & Validation

The system underwent rigorous testing using a dedicated test script. The following scenarios were validated:

Test Scenario	Description	Outcome
Integration Check	Loading <code>models/model-best</code> from Milestone 1.	Pass: Model loaded successfully.
Context Test	User initiates transfer -> Bot asks "How much?" -> User provides amount.	Pass: Logic correctly mapped "500" to the amount slot.
Menu Navigation	User selects "Loan" -> "Car Loan".	Pass: Correct leaf-node response retrieved.
FAQ Fallback	User asks "What is my balance?" -> Bot requests account number.	Pass: CSV response triggered correctly; subsequent input captured via Regex.

5. Challenges & Solutions

Challenge 1: Context Amnesia

- *Issue:* When the bot asked "How much?", and the user replied "500", the NLU re-processed "500" as a new command, often failing or misclassifying it.

- *Solution:* Implemented **State Priority**. The `handle_turn` function now checks `current_state` before calling the NLU. If the state is `WAITING`, the NLU is ignored in favor of data extraction.

Challenge 2: Data Quality (NaN Errors)

- *Issue:* The system crashed when accessing empty rows in the `training_and_responses.csv`.
 - *Solution:* Implemented robust Pandas data cleaning: `df = df.dropna(subset=[ 'response' ])` to ensure only valid QA pairs are loaded.
-

6. Future Roadmap (Milestone 3)

With the Dialogue Logic (The "Brain") and NLU (The "Ears") now fully functional and integrated, the final phase will focus on **Connectivity**.

- **Database Integration:** Replace the simulated `transfer_money` success message with a real SQL/SQLite transaction.
- **API Connection:** Connect `check_balance` to a mock banking API to fetch dynamic user data.
- **Deployment:** Containerize the application for web or API deployment.